

CLOUD INFRASTRUCTURE AUTOMATION USING TERRAFORM AND AWS

Services: EC2, RDS, VPC, Terraform

Description: Automate the provisioning of an application stack with Terraform. The stack includes EC2 instances in a VPC with a public and private subnet, a load balancer, and an RDS database.

Skills: Infrastructure as Code (IaC), Terraform, VPC setup.

Definition: This project automates the deployment of AWS cloud infrastructure using Terraform. It creates resources including a VPC, subnets, EC2 instances, an Application Load Balancer, and an RDS database, enabling infrastructure management through code rather than manual setup.

Scope of the Project: The scope of this project is to design and deploy a secure and scalable AWS cloud infrastructure using Terraform. It includes setting up networking components, compute resources, load balancing, and a database environment. The project demonstrates how Infrastructure as Code (IaC) can automate resource provisioning, reduce manual effort, and ensure consistent infrastructure deployment.

Methodologies:

1. Infrastructure as Code (IaC) using Terraform – Best Method

- Uses Terraform to automate AWS infrastructure deployment.
- Infrastructure is defined and managed through code.
- Supports version control and reusable configurations.
- Reduces deployment time and configuration errors.
- Enables repeatable and consistent infrastructure creation.
- Suitable for enterprise and production environments.

2. Manual Configuration using AWS Console – Alternative Method

- Resources are created manually through the AWS Management Console.
- Suitable only for small-scale environments.
- Requires significant manual effort.
- Difficult to maintain consistency across environments.
- Prone to configuration mistakes and operational overhead.
- Not suitable for large-scale infrastructure deployments.

Services Used in this Project:

1. Amazon EC2 (Elastic Compute Cloud)

Definition: Amazon EC2 is a cloud service that provides virtual servers to run applications in the AWS environment.

Purpose: EC2 is used to host and run the web application inside the VPC.

Role: EC2 acts as the application server that receives traffic from the load balancer and processes user requests.

2. RDS (Relational Database Service)

Definition: Amazon RDS is a managed database service used to create, operate, and scale relational databases in the AWS cloud.

Purpose: RDS is used to store and manage the application's data securely in the private subnet.

Role: RDS acts as the backend database that stores and retrieves data for the application running on the EC2 instance.

3. VPC (Virtual Private Cloud)

Definition: Amazon VPC is a virtual private network in AWS that allows you to securely host and manage cloud resources.

Purpose: VPC is used to create an isolated and secure network environment for deploying the application infrastructure.

Role: VPC acts as the main network that connects and controls communication between EC2, subnets, load balancer, and RDS.

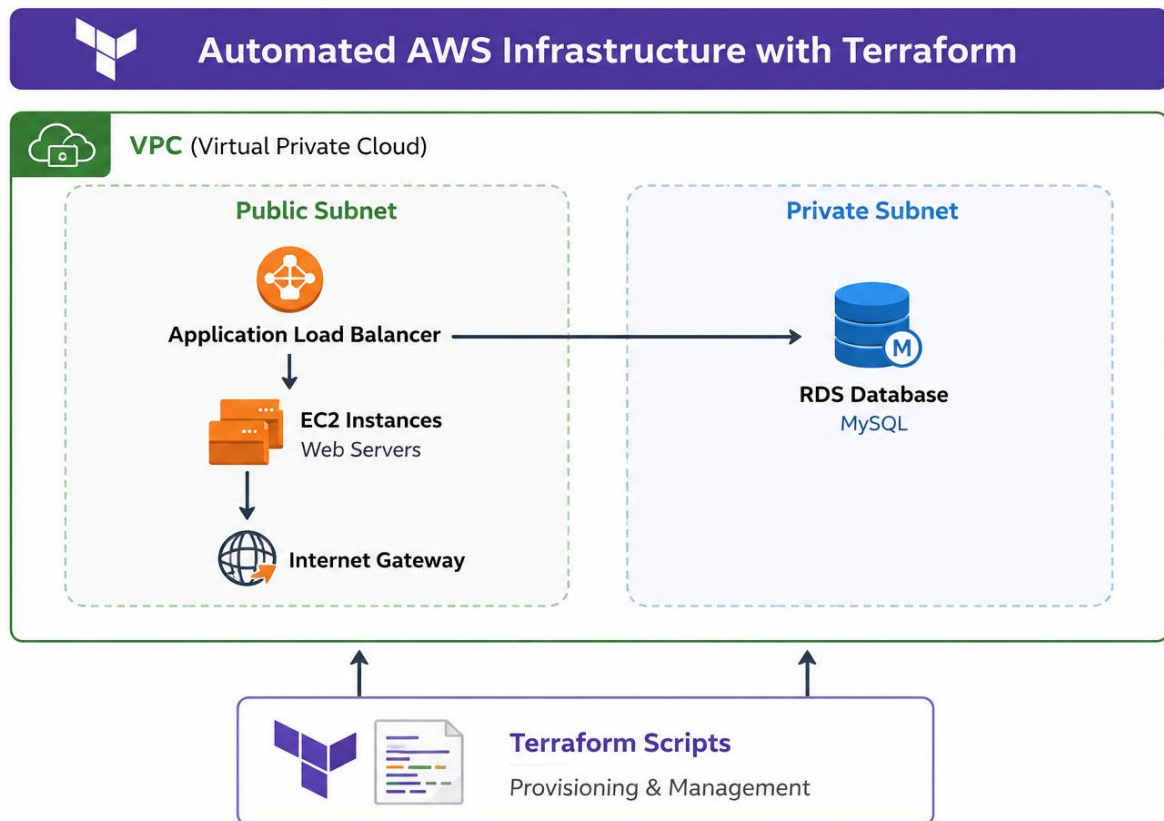
4. Terraform

Definition: Terraform is an Infrastructure as Code (IaC) tool used to create and manage cloud resources using configuration files.

Purpose: Terraform is used to automate the provisioning of AWS infrastructure such as VPC, EC2, load balancer, and RDS.

Role: Terraform acts as the automation tool that deploys and manages the entire application infrastructure through code.

Architecture Diagram:



Implementation

STEP 1: Launch an EC2 Server

- Go to AWS Console → EC2 → Launch Instance.
- Choose Amazon Linux 2023 AMI.
- Instance Type: t3. micro.
- Storage: 20 GB.
- Region: Mumbai (ap-south-1).
- Configure Security Group.
- Create or Select Key Pair.
- Launch Instance.

Instance summary for i-0aef80d86eeb10c33 (terraform)

Updated less than a minute ago

Instance ID i-0aef80d86eeb10c33	Public IPv4 address 13.207.193.168 open address	Private IPv4 addresses 172.31.43.183
IPv6 address -	Instance state Running	Public DNS ec2-13-207-193-168.ap-south-1.compute.amazonaws.com open address
Hostname type IP name: ip-172-31-43-183.ap-south-1.compute.internal	Private IP DNS name (IPv4 only) ip-172-31-43-183.ap-south-1.compute.internal	Elastic IP addresses -
Answer private resource DNS name IPv4 (A)	Instance type t3.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address 13.207.193.168 [Public IP]	VPC ID vpc-0604cab257558139c	Auto Scaling Group name -
IAM role -	Subnet ID subnet-0474e24d37329b5ab	Managed false
IMDSv2 Required	Instance ARN arn:aws:ec2:ap-south-1:479897913078:instance/i-0aef80d86eeb10c33	
Operator -		

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

STEP :2 Install and Check the Terraform version

- Connect to the EC2 instance.
- Download Terraform package.
- Install Terraform.
- Verify installation:
- terraform -version

```
[ec2-user@ip-172-31-43-183 ~]$ terraform -version
Terraform v1.5.5
on linux amd64
[ec2-user@ip-172-31-43-183 ~]$ aws --version
aws-cli/2.33.15 Python/3.9.25 Linux/6.1.172-216.329.amzn2023.x86_64 source/x86_64.amzn.2023
[ec2-user@ip-172-31-43-183 ~]$ aws configure
AWS Access Key ID [*****]: AWS Secret Access Key [*****dPUC]: Default region name [ap-south-1]: Default output format [None]: [ec2-user@ip-172-31-43-183 ~]$
[ec2-user@ip-172-31-43-183 ~]$ cd terraform-aws-project/
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ cd ..
[ec2-user@ip-172-31-43-183 ~]$ ls
terraform-aws-project
[ec2-user@ip-172-31-43-183 ~]$ cd terraform-aws-project/
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ nano main.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ nano variable.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf variable.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ nano terraform.tfvars
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf terraform.tfvars variable.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ nano output.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf output.tf terraform.tfvars variable.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.47.0...
- Installed hashicorp/aws v6.47.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.
```

i-0aef80d86eeb10c33 (terraform)

PublicIPs: 13.207.193.168 PrivateIPs: 172.31.43.183

STEP3: Check aws version

- Check AWS CLI version:
- aws --version

```

[ec2-user@ip-172-31-43-183 ~]$ aws --version
aws-cli/2.33.15 Python/3.9.25 Linux/6.1.172-216.329.amzn2023.x86_64 source/x86_64.amzn.2023
[ec2-user@ip-172-31-43-183 ~]$ aws configure
AWS Access Key ID [*****2R5I]: AWS Secret Access Key [*****dEUC]: Default region name [ap-south-1]: Default output format [None]: [ec2-user@ip-172-31-43-183 ~]$
[ec2-user@ip-172-31-43-183 ~]$ mkdir terraform-aws-project
[ec2-user@ip-172-31-43-183 ~]$ cd terraform-aws-project/
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ cd ..
[ec2-user@ip-172-31-43-183 ~]$ ls
terraform-aws-project
[ec2-user@ip-172-31-43-183 ~]$ cd terraform-aws-project/
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ nano main.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ nano variable.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf variable.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ nano terraform.tfvars
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf terraform.tfvars variable.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ nano output.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf output.tf terraform.tfvars variable.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.47.0...
- Installed hashicorp/aws v6.47.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see

```

i-0aef80d86eeb10c33 (terraform)

PublicIPs: 13.207.193.168 PrivateIPs: 172.31.43.183

STEP 4: Create Access Keys and Configure AWS CLI

- Go to IAM → Users.
- Create Access Key.
- Configure AWS CLI:

aws configure

➤ Enter:

- Access Key ID
- Secret Access Key
- Region
- Output Format

```

[ec2-user@ip-172-31-43-183 ~]$ aws configure
AWS Access Key ID [*****2R5I]: AWS Secret Access Key [*****dEUC]: Default region name [ap-south-1]: Default output format [None]: [ec2-user@ip-172-31-43-183 ~]$
[ec2-user@ip-172-31-43-183 ~]$ mkdir terraform-aws-project
[ec2-user@ip-172-31-43-183 ~]$ cd terraform-aws-project/
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ cd ..
[ec2-user@ip-172-31-43-183 ~]$ ls
terraform-aws-project
[ec2-user@ip-172-31-43-183 ~]$ cd terraform-aws-project/
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ nano main.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ nano variable.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf variable.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ nano terraform.tfvars
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf terraform.tfvars variable.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ nano output.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf output.tf terraform.tfvars variable.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.47.0...
- Installed hashicorp/aws v6.47.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

```

i-0aef80d86eeb10c33 (terraform)

PublicIPs: 13.207.193.168 PrivateIPs: 172.31.43.183

STEP 5: Created a project directory and added four Terraform file .Create the project folder and Terraform files:

- main.tf
- variables.tf
- terraform.tfvars
- outputs.tf

```
[ec2-user@ip-172-31-43-183 ~]$ mkdir terraform-aws-project
[ec2-user@ip-172-31-43-183 ~]$ cd terraform-aws-project/
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ cd ..
[ec2-user@ip-172-31-43-183 ~]$ ls
terraform-aws-project
[ec2-user@ip-172-31-43-183 ~]$ cd terraform-aws-project/
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ nano main.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ nano variable.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf  variable.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ nano terraform.tfvars
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf  terraform.tfvars  variable.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ nano output.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ ls
main.tf  output.tf  terraform.tfvars  variable.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.47.0...
- Installed hashicorp/aws v6.47.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
```

i-0aef80d86eeb10c33 (terraform)
PublicIPs: 13.207.193.168 PrivateIPs: 172.31.43.183

STEP 6: main.tf

- Configure VPC.
- Configure Subnets.
- Configure Internet Gateway.
- Configure Route Tables.
- Configure Security Groups.
- Configure EC2 Instance.
- Configure Application Load Balancer.
- Configure RDS Database.

```

GNU nano 8.3          main.tf          Modified
# provider "aws" {
#   region = var.region
# }

# VPC
resource "aws_vpc" "main_vpc" {
  cidr_block = "10.0.0.0/16"

  tags = {
    Name = "Project-VPC"
  }
}

# Public Subnet 1
resource "aws_subnet" "public_subnet_1" {
  vpc_id      = aws_vpc.main_vpc.id
  cidr_block  = "10.0.1.0/24"
  availability_zone = "ap-south-1a"
  map_public_ip_on_launch = true

  tags = {
    Name = "Public-Subnet-1"
  }
}

# Public Subnet 2
resource "aws_subnet" "public_subnet_2" {
  vpc_id      = aws_vpc.main_vpc.id
  cidr_block  = "10.0.2.0/24"
  availability_zone = "ap-south-1b"
  map_public_ip_on_launch = true

  tags = {
    Name = "Public-Subnet-2"
  }
}

```

i-0aef80d86eeb10c33 (terraform)

PublicIPs: 13.207.193.168 PrivateIPs: 172.31.43.183

STEP 7: variable.tf

- Define Terraform variables.
- Store reusable configuration values.

```

GNU nano 8.3          variable.tf      Modified
variable "region" {
  description = "AWS Region"
}

variable "ami_id" {
  description = "AMI ID for EC2"
}

variable "instance_type" {
  description = "EC2 Instance Type"
}

variable "key_name" {
  description = "AWS Key Pair Name"
}

variable "instance_name" {
  description = "EC2 Instance Name"
}

```

i-0aef80d86eeb10c33 (terraform)

PublicIPs: 13.207.193.168 PrivateIPs: 172.31.43.183

STEP 8: terraform. tfvars

- Assign values to variables.
- Customize infrastructure settings.

```
GNU nano 8.3 terraform.tfvars Modified
region = "ap-south-1"
ami_id = "ami-0685bcc683dadb6b9"
instance_type = "t3.micro"
key_name = "aws-keypair"
instance_name = "Terraform-EC2-Server"
```

i-0aef80d86eeb10c33 (terraform)

PublicIPs: 13.207.193.168 PrivateIPs: 172.31.43.183

STEP 9: output.tf

- Display resource IDs.
- Display Load Balancer DNS.
- Display RDS Endpoint.

```
GNU nano 8.3 output.tf Modified
output "instance_id" {
  value = aws_instance.web_server.id
}

output "public_ip" {
  value = aws_instance.web_server.public_ip
}

output "private_ip" {
  value = aws_instance.web_server.private_ip
}

output "rds_endpoint" {
  value = aws_db_instance.db.endpoint
}

output "load_balancer_dns" {
  value = aws_lb.project_alb.dns_name
}
```

i-0aef80d86eeb10c33 (terraform)

PublicIPs: 13.207.193.168 PrivateIPs: 172.31.43.183

STEP 10: Initialize Terraform

- Run:
- terraform init
- Downloads required providers.
- Initializes Terraform workspace.


```
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ terraform init
Initializing provider plugins found in the configuration...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.47.0...
- Installed hashicorp/aws v6.47.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ terraform fmt
main.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ terraform validate
Success! The configuration is valid.

[ec2-user@ip-172-31-43-183 terraform-aws-project]$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_db_instance.db will be created
+ resource "aws_db_instance" "db" {
  + address              = (known after apply)
  + allocated_storage    = 20
  + apply_immediately    = false
  + arn                  = (known after apply)
  + auto_minor_version_upgrade = true
  + availability_zone     = (known after apply)
  + backup_retention_period = (known after apply)
  + backup_target         = (known after apply)
  + backup_window         = (known after apply)
  + ca_cert_identifier    = (known after apply)
  + character_set_name    = (known after apply)
  + copy_tags_to_snapshot = false
  + database_insights_mode = (known after apply)
  + db_name              = "projectdb"
  + db_subnet_group_name = "project-db-subnet-group"
  + dedicated_log_volume = false
  + delete_automated_backups = true
  + domain_fqdn          = (known after apply)
  + endpoint              = (known after apply)
  + engine                = "mysql"
  + engine_lifecycle_support = (known after apply)
  + engine_version        = "8.0"
  + engine_version_actual  = (known after apply)
  + hosted_zone_id        = (known after apply)
  + id                   = (known after apply)
```

i-0aef80d86eeb10c33 (terraform)
PublicIPs: 13.207.193.168 PrivateIPs: 172.31.43.183

STEP 11: Format Terraform Configuration

- Run
- terraform fmt
- Formats Terraform files.

```
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ terraform fmt
main.tf
[ec2-user@ip-172-31-43-183 terraform-aws-project]$ terraform validate
Success! The configuration is valid.

[ec2-user@ip-172-31-43-183 terraform-aws-project]$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_db_instance.db will be created
+ resource "aws_db_instance" "db" {
  + address              = (known after apply)
  + allocated_storage    = 20
  + apply_immediately    = false
  + arn                  = (known after apply)
  + auto_minor_version_upgrade = true
  + availability_zone     = (known after apply)
  + backup_retention_period = (known after apply)
  + backup_target         = (known after apply)
  + backup_window         = (known after apply)
  + ca_cert_identifier    = (known after apply)
  + character_set_name    = (known after apply)
  + copy_tags_to_snapshot = false
  + database_insights_mode = (known after apply)
  + db_name              = "projectdb"
  + db_subnet_group_name = "project-db-subnet-group"
  + dedicated_log_volume = false
  + delete_automated_backups = true
  + domain_fqdn          = (known after apply)
  + endpoint              = (known after apply)
  + engine                = "mysql"
  + engine_lifecycle_support = (known after apply)
  + engine_version        = "8.0"
  + engine_version_actual  = (known after apply)
  + hosted_zone_id        = (known after apply)
  + id                   = (known after apply)
```

i-0aef80d86eeb10c33 (terraform)
PublicIPs: 13.207.193.168 PrivateIPs: 172.31.43.183

STEP 12 : Validate Terraform Configuration

- Run:
- terraform validate
- Checks configuration syntax and dependencies.

```

ec2-user@ip-172-31-43-183 terraform-aws-project]$ terraform validate
Success! The configuration is valid.

ec2-user@ip-172-31-43-183 terraform-aws-project]$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_db_instance.db will be created
+ resource "aws_db_instance" "db" {
  + address                = (known after apply)
  + allocated_storage      = 20
  + apply_immediately      = false
  + arn                    = (known after apply)
  + auto_minor_version_upgrade = true
  + availability_zone       = (known after apply)
  + backup_retention_period = (known after apply)
  + backup_target           = (known after apply)
  + backup_window           = (known after apply)
  + ca_cert_identifier      = (known after apply)
  + character_set_name      = (known after apply)
  + copy_tags_to_snapshot   = false
  + database_insights_mode  = (known after apply)
  + db_name                 = "projectdb"
  + db_subnet_group_name    = "project-db-subnet-group"
  + dedicated_log_volume    = false
  + delete_automated_backups = true
  + domain_fqdn             = (known after apply)
  + endpoint                = (known after apply)
  + engine                   = "mysql"
  + engine_lifecycle_support = (known after apply)
  + engine_version          = "8.0"
  + engine_version_actual    = (known after apply)
  + hosted_zone_id          = (known after apply)
  + id                      = (known after apply)
  + identifier               = "projectdb"
  + identifier_prefix        = (known after apply)

```

i-0aef80d86eeb10c33 (terraform)

PublicIPs: 13.207.193.168 PrivateIPs: 172.31.43.183

STEP 13: Check Infrastructure Plan

- Run:
- terraform plan
- Displays resources that will be created.

```

[ec2-user@ip-172-31-43-183 terraform-aws-project]$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_db_instance.db will be created
+ resource "aws_db_instance" "db" {
  + address                = (known after apply)
  + allocated_storage      = 20
  + apply_immediately      = false
  + arn                    = (known after apply)
  + auto_minor_version_upgrade = true
  + availability_zone       = (known after apply)
  + backup_retention_period = (known after apply)
  + backup_target           = (known after apply)
  + backup_window           = (known after apply)
  + ca_cert_identifier      = (known after apply)
  + character_set_name      = (known after apply)
  + copy_tags_to_snapshot   = false
  + database_insights_mode  = (known after apply)
  + db_name                 = "projectdb"
  + db_subnet_group_name    = "project-db-subnet-group"
  + dedicated_log_volume    = false
  + delete_automated_backups = true
  + domain_fqdn             = (known after apply)
  + endpoint                = (known after apply)
  + engine                   = "mysql"
  + engine_lifecycle_support = (known after apply)
  + engine_version          = "8.0"
  + engine_version_actual    = (known after apply)
  + hosted_zone_id          = (known after apply)
  + id                      = (known after apply)
  + identifier               = "projectdb"
  + identifier_prefix        = (known after apply)
  + instance_class           = "db.t3.micro"
  + iops                    = (known after apply)
  + kms_key_id              = (known after apply)

```

i-0aef80d86eeb10c33 (terraform)

PublicIPs: 13.207.193.168 PrivateIPs: 172.31.43.183

```

+ resource "aws_vpc" "main_vpc" {
+   arn                                = (known after apply)
+   cidr_block                         = "10.0.0.0/16"
+   default_network_acl_id             = (known after apply)
+   default_route_table_id             = (known after apply)
+   default_security_group_id          = (known after apply)
+   dhcp_options_id                    = (known after apply)
+   enable_dns_hostnames                = (known after apply)
+   enable_dns_support                  = true
+   enable_network_address_usage_metrics = (known after apply)
+   id                                 = (known after apply)
+   instance_tenancy                    = "default"
+   ipv6_association_id                 = (known after apply)
+   ipv6_cidr_block                     = (known after apply)
+   ipv6_cidr_block_network_border_group = (known after apply)
+   main_route_table_id                 = (known after apply)
+   owner_id                           = (known after apply)
+   region                             = "ap-south-1"
+   tags                               = {
+     "Name" = "Project-VPC"
+   }
+   tags_all                           = {
+     "Name" = "Project-VPC"
+   }
+ }

```

Plan: 16 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```

+ instance_id           = (known after apply)
+ load_balancer_dns     = (known after apply)
+ private_ip            = (known after apply)
+ public_ip             = (known after apply)
+ rds_endpoint          = (known after apply)

```

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

[ec2-user@ip-172-31-43-183 terraform-aws-project]\$

i-0aef80d86eeb10c33 (terraform)

PublicIPs: 13.207.193.168 PrivateIPs: 172.31.43.183

STEP 14: Deploy Infrastructure

- Run: terraform apply
- Review execution plan.
- Type: yes
- Deploy resources.

[ec2-user@ip-172-31-43-183 terraform-aws-project]\$ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

aws_db_instance.db will be created

```

+ resource "aws_db_instance" "db" {
+   address                        = (known after apply)
+   allocated_storage              = 20
+   apply_immediately              = false
+   arn                            = (known after apply)
+   auto_minor_version_upgrade     = true
+   availability_zone              = (known after apply)
+   backup_retention_period         = (known after apply)
+   backup_target                  = (known after apply)
+   backup_window                  = (known after apply)
+   ca_cert_identifier              = (known after apply)
+   character_set_name              = (known after apply)
+   copy_tags_to_snapshot          = false
+   database_insights_mode         = (known after apply)
+   db_name                        = "projectdb"
+   db_subnet_group_name           = "project-db-subnet-group"
+   dedicated_log_volume           = false
+   delete_automated_backups       = true
+   domain_fqdn                    = (known after apply)
+   endpoint                      = (known after apply)
+   engine                         = "mysql"
+   engine_lifecycle_support        = (known after apply)
+   engine_version                 = "8.0"
+   engine_version_actual           = (known after apply)
+   hosted_zone_id                 = (known after apply)
+   id                             = (known after apply)
+   identifier                     = "projectdb"
+   identifier_prefix               = (known after apply)
+   instance_class                 = "db.t3.micro"
+   logs                           = (known after apply)
+   kms_key_id                     = (known after apply)

```

i-0aef80d86eeb10c33 (terraform)

PublicIPs: 13.207.193.168 PrivateIPs: 172.31.43.183

```

+ resource "aws_vpc" "main_vpc" {
  + arn                        = (known after apply)
  + cidr_block                 = "10.0.0.0/16"
  + default_network_acl_id    = (known after apply)
  + default_route_table_id    = (known after apply)
  + default_security_group_id = (known after apply)
  + dhcp_options_id           = (known after apply)
  + enable_dns_hostnames      = (known after apply)
  + enable_dns_support         = true
  + enable_network_address_usage_metrics = (known after apply)
  + id                        = (known after apply)
  + instance_tenancy          = "default"
  + ipv6_association_id        = (known after apply)
  + ipv6_cidr_block            = (known after apply)
  + ipv6_cidr_block_network_border_group = (known after apply)
  + main_route_table_id        = (known after apply)
  + owner_id                   = (known after apply)
  + region                     = "ap-south-1"
  + tags                       = {
    + "Name" = "Project-VPC"
  }
  + tags_all                   = {
    + "Name" = "Project-VPC"
  }
}

```

Plan: 16 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```

+ instance_id           = (known after apply)
+ load_balancer_dns      = (known after apply)
+ private_ip             = (known after apply)
+ public_ip             = (known after apply)
+ rds_endpoint          = (known after apply)

```

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

i-Oaef80d86eeb10c33 (terraform)

PublicIPs: 13.207.193.168 PrivateIPs: 172.31.43.183

```

aws_db_instance.db: Still creating... [01m40s elapsed]
aws_lb_project_alb: Still creating... [01m50s elapsed]
aws_db_instance.db: Still creating... [01m50s elapsed]
aws_lb_project_alb: Still creating... [02m00s elapsed]
aws_db_instance.db: Still creating... [02m00s elapsed]
aws_lb_project_alb: Still creating... [02m10s elapsed]
aws_db_instance.db: Still creating... [02m10s elapsed]
aws_lb_project_alb: Still creating... [02m20s elapsed]
aws_db_instance.db: Still creating... [02m20s elapsed]
aws_lb_project_alb: Still creating... [02m30s elapsed]
aws_db_instance.db: Still creating... [02m30s elapsed]
aws_lb_project_alb: Still creating... [02m40s elapsed]
aws_db_instance.db: Still creating... [02m40s elapsed]
aws_lb_project_alb: Still creating... [02m50s elapsed]
aws_db_instance.db: Still creating... [02m50s elapsed]
aws_lb_project_alb: Creation complete after 2m51s [id=arn:aws:elasticloadbalancing:ap-south-1:479897913078:loadbalancer/app/project-alb/badaeb865dd0b860]
aws_lb_listener.listener: Creating...
aws_lb_listener.listener: Creation complete after 0s [id=arn:aws:elasticloadbalancing:ap-south-1:479897913078:listener/app/project-alb/badaeb865dd0b860/1f6b087c27b53b84]
aws_db_instance.db: Still creating... [03m00s elapsed]
aws_db_instance.db: Still creating... [03m10s elapsed]
aws_db_instance.db: Still creating... [03m20s elapsed]
aws_db_instance.db: Still creating... [03m30s elapsed]
aws_db_instance.db: Still creating... [03m40s elapsed]
aws_db_instance.db: Still creating... [03m50s elapsed]
aws_db_instance.db: Still creating... [04m00s elapsed]
aws_db_instance.db: Still creating... [04m10s elapsed]
aws_db_instance.db: Still creating... [04m20s elapsed]
aws_db_instance.db: Still creating... [04m30s elapsed]
aws_db_instance.db: Creation complete after 4m34s [id=db-JT4LVS6UVCB52T3CHVRDHYOV4E]

```

Apply complete! Resources: 16 added, 0 changed, 0 destroyed.

Outputs:

```

instance_id = "i-08f6b80c1c8cfadbe"
load_balancer_dns = "project-alb-1874621532.ap-south-1.elb.amazonaws.com"
private_ip = "10.0.1.118"
public_ip = "13.206.80.106"
rds_endpoint = "projectdb.cjaocmm6kkdm.ap-south-1.rds.amazonaws.com:3306"
[ec2-user@ip-172-31-43-183 terraform-aws-project]$

```

i-Oaef80d86eeb10c33 (terraform)

PublicIPs: 13.207.193.168 PrivateIPs: 172.31.43.183

STEP 15: Verify Resources in AWS Console → EC2 (Elastic Compute Cloud)

- Open AWS Console → EC2.
- Verify EC2 instance creation.
- Check instance status.

EC2 > Instances > i-08f6b80c1c8cfadbe

Instance summary for i-08f6b80c1c8cfadbe (Terraform-EC2-Server) [info](#)

Updated less than a minute ago

Instance ID i-08f6b80c1c8cfadbe	Public IPv4 address 13.206.80.106 open address	Private IPv4 addresses 10.0.1.118
IPv6 address -	Instance state Running	Public DNS -
Hostname type IP name: ip-10-0-1-118.ap-south-1.compute.internal	Private IP DNS name (IPv4 only) ip-10-0-1-118.ap-south-1.compute.internal	Elastic IP addresses -
Answer private resource DNS name -	Instance type t3.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address 13.206.80.106 [Public IP]	VPC ID vpc-00874d964c14fa242 (Project-VPC)	Auto Scaling Group name -
IAM role -	Subnet ID subnet-0ff9f43b6ff67d62e (Public-Subnet-1)	Managed false
IMDSv2 Required	Instance ARN arn:aws:ec2:ap-south-1:479897913078:instance/i-08f6b80c1c8cfadbe	
Operator -		

[Details](#) | [Status and alarms](#) | [Monitoring](#) | [Security](#) | [Networking](#) | [Storage](#) | [Tags](#)

▼ **Instance details** [info](#)

STEP 16: VPC (Virtual Private Cloud)

- Open AWS Console → VPC.
- Verify VPC creation.
- Verify networking configuration.
-

VPC > Your VPCs > vpc-00874d964c14fa242

VPC dashboard < vpc-00874d964c14fa242 / Project-VPC [Actions](#)

AWS Global View [Filter by VPC:](#)

▼ **Virtual private cloud**

- [Your VPCs](#)
- [Subnets](#)
- [Route tables](#)
- [Internet gateways](#)
- [Egress-only Internet gateways](#)
- [DHCP option sets](#)
- [Elastic IPs](#)
- [Managed prefix lists](#)
- [NAT gateways](#)
- [Peering connections](#)
- [Route servers](#)

▼ **Security**

- [Network ACLs](#)
- [Security groups](#)

▼ **PrivateLink and Lattice**

- [Getting started](#)
- [Endpoints](#)
- [Endpoint services](#)
- [Service networks](#)

Details [info](#)

VPC ID vpc-00874d964c14fa242	State Available	Block Public Access Off	DNS hostnames Disabled
DNS resolution Enabled	Tenancy default	DHCP option set dopt-080425edd2d247315	Main route table rtb-07997a8c9c27bf31f
Main network ACL acl-0819911509b709664	Default VPC No	IPv4 CIDR 10.0.0.0/16	IPv6 pool -
IPv6 CIDR (Network border group) -	Network Address Usage metrics Disabled	Route 53 Resolver DNS Firewall rule groups -	Owner ID 479897913078
Encryption control ID -	Encryption control mode -		

[Resource map](#) | [CIDRs](#) | [Flow logs](#) | [Tags](#) | [Integrations](#)

Resource map [info](#) [Show all details](#)

VPC: Your AWS virtual network

Project-VPC

Subnets (3)
Subnets within this VPC

- ap-south-1a
 - Public-Subnet-1
 - Private-Subnet
- ap-south-1b

Route tables (2)
Route network traffic to resources

- rtb-07997a8c9c27bf31f
- rtb-0e2a923738a5fbfaf

Network Connections (1)
Connections to other networks

- Project-IGW

STEP 17: Public Subnet and Private Subnet

- Confirm subnet creation.
- Verify route table associations.

VPC > Subnets

VPC dashboard

AWS Global View

Filter by VPC:

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only Internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Peering connections

Route servers

Security

Network ACLs

Security groups

PrivateLink and Lattice

Getting started

Endpoints

Endpoint services

Service networks

Subnets (6) Info

Find subnets by attribute or tag

Name

Subnet ID

State

VPC

Block Public...

IPv4 CIDR

IPv6 CIDR

Public-Subnet-1

subnet-0f9f43b6ff67d62e

Available

vpc-00874d964c14fa242 | Proj...

Off

10.0.1.0/24

-

Public-Subnet-2

subnet-05ec348211fac3933

Available

vpc-00874d964c14fa242 | Proj...

Off

10.0.2.0/24

-

Private-Subnet

subnet-0ccc8574a193da50e

Available

vpc-00874d964c14fa242 | Proj...

Off

10.0.3.0/24

-

-

subnet-0b60b5d0720ffa1a0

Available

vpc-0604cab257558139c

Off

172.31.0.0/20

-

-

subnet-0474e24d37329b5ab

Available

vpc-0604cab257558139c

Off

172.31.32.0/20

-

-

subnet-058f7538934d7e57e

Available

vpc-0604cab257558139c

Off

172.31.16.0/20

-

Select a subnet

STEP 18: RDS (Relational Database Management)

- Open AWS Console → RDS.
- Verify database creation.
- Verify endpoint availability.

Aurora and RDS > Databases > projectdb

Aurora and RDS

Dashboard

Databases

Performance insights

Snapshots

Exports in Amazon S3

Automated backups

Reserved instances

Proxies

Subnet groups

Parameter groups

Option groups

Custom engine versions

Zero-ETL integrations

Events

Event subscriptions

Recommendations 0

Certificate update

projectdb

Summary

DB identifier
projectdb

CPU
5.93%

Status
Available

Class
db.t3.micro

Role
Instance

Current activity
0 Connections

Engine
MySQL Community

Region & AZ
ap-south-1b

Recommendations

Connectivity & security

Monitoring

Logs & events

Configuration

Zero-ETL integrations

Maintenance & backups

Data migrations

Tags

R

Connect using

Code snippets

CloudShell

Endpoints

Internet access gateway

IAM Authentication

Programming language

Endpoint type

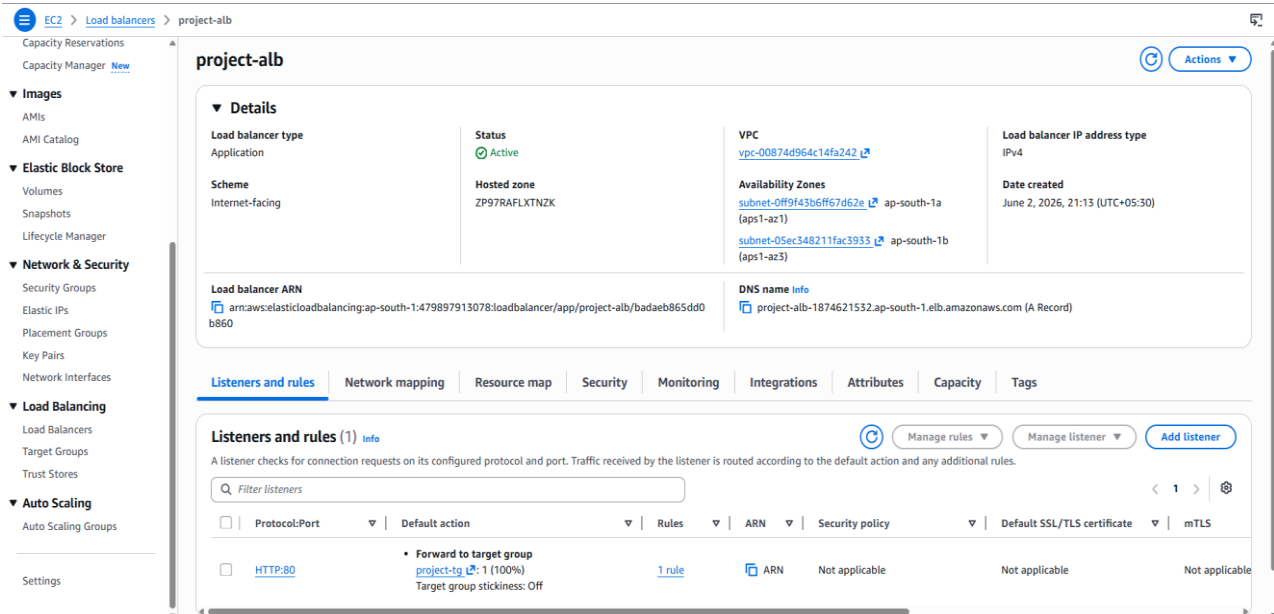
Connection steps

1 curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem

1 mysql -h projectdb.cjaocm6kkdm.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p --ssl-mode=VERIFY_IDENTITY --ssl-ca=./global-bundle.pem

STEP 19: LB (Load Balancer)

- Open AWS Console → EC2 → Load Balancers.
- Verify ALB creation.
- Check target group health status.



Troubleshooting:

Issue: Terraform Deployment Failure

Causes:

- Incorrect AWS credentials.
- Syntax errors in Terraform configuration files.
- Resource name conflicts.
- Existing AWS resources with the same name.

Solution:

- Verify AWS credentials using AWS CLI.
- Run terraform validate to check configuration errors.
- Review Terraform error messages and logs.
- Use unique resource names or import existing resources into Terraform state.

Result:

After correcting the configuration and validating the Terraform files, the infrastructure was successfully deployed on AWS.

Conclusion:

This project successfully demonstrated the automation of cloud infrastructure deployment using Terraform on AWS. By implementing Infrastructure as Code (IaC), resources such as VPC, subnets, EC2 instances, load balancer, and RDS database were provisioned automatically rather than being configured manually. The project provided practical knowledge of AWS networking, compute services, and database management. It also highlighted how automation enhances consistency, scalability, and operational efficiency in cloud environments. Overall, this project emphasizes the significance of Terraform in modern DevOps workflows and cloud infrastructure management.